

Environmental Statistics

Lecture 9: Bootstrapping (and jackknife)

Ikedichi Edward Azuh

Carsten Dormann

1. Overview

In environmental and biological statistics, we are often interested in estimating quantities such as means, regression coefficients, or more complex indices derived from data. However, in most real-world settings the true sampling distribution of these estimators is unknown or analytically intractable.

Classical approaches to uncertainty quantification rely on strong assumptions (e.g. normality, independence, or linear approximations via Taylor expansion). These assumptions are frequently violated in environmental data, especially in the presence of non-linearity, spatial dependence, or complex summary statistics.

This lecture introduces **resampling-based methods for uncertainty quantification**, focusing on bootstrapping as a practical and widely applicable alternative to analytical error propagation.

We also briefly introduce the jackknife as a related, simpler resampling technique.

In this lecture we cover:

- The need for uncertainty quantification in statistical estimation
- Classical error propagation and its limitations (Taylor approximation)
- The bootstrap: idea, algorithm, and interpretation
- Bootstrap-based standard errors and confidence intervals
- Bootstrap applications: means, regression models, and complex indices
- Limitations of the bootstrap
- The jackknife: leave-one-out resampling and variance estimation
- Comparison of uncertainty estimates from OLS, jackknife, and bootstrap
- Practical implementation in R using for-loops and simulation-based inference

2. Why do we need bootstrapping?

In many statistical applications, we estimate a quantity of interest (mean, regression coefficient, or derived index), but the sampling distribution of this estimator is unknown.

Classical statistical inference relies on:

- Large-sample approximations (Central Limit Theorem)
- Strong distributional assumptions (e.g. normality)
- Simplified variance formulas

However, in environmental and biological systems:

- models are often nonlinear

- data may be dependent or heterogeneous
- derived statistics can be highly complex

As a result, analytical formulas for uncertainty often become unreliable or unavailable.

This motivates a **data-driven approach to uncertainty estimation**.

3. Classical uncertainty propagation

A traditional way to quantify uncertainty is error propagation. The basic idea is to estimate how uncertainty in the input variables affects the uncertainty of the final result.

Suppose the quantity of interest is a function of several inputs:

$$I = f(x_1, x_2, \dots, x_p).$$

If the input uncertainties are relatively small and the function is sufficiently smooth, f can be approximated by a first-order Taylor expansion around the observed input values. Under this approximation, the variance of the output is

$$\text{Var}(I) \approx \nabla f^\top \Sigma \nabla f,$$

where

- ∇f is the gradient (i.e., the vector of partial derivatives) of f , describing how sensitive the output is to each input variable.
- Σ is the covariance matrix of the input variables, describing both their individual uncertainties and their correlations.

Limitations:

The first-order approximation works well when the function is approximately linear near the point of interest. However, it can become inaccurate when

- $f(\cdot)$ is highly nonlinear,
- $f(\cdot)$ contains discontinuities (e.g., minimum or maximum operators),
- $f(\cdot)$ is evaluated through simulations or iterative algorithms, making derivatives difficult or impossible to compute, or the covariance structure of the inputs is unknown or difficult to estimate.

3.1.0 data-driven example

We consider a function of three measured inputs:

$$I = f(x_1, x_2, x_3).$$

In practice, the inputs are not fixed values but are observed repeatedly. This allows us to estimate both the **expected values** and the **uncertainty structure** directly from data.

3.1.1 Data (repeated measurements)

```
data <- data.frame(
  x1 = c(2.0, 2.1, 1.9, 2.2, 2.0, 2.1),
  x2 = c(5.1, 5.0, 5.2, 4.9, 5.0, 5.1),
  x3 = c(3.2, 3.1, 3.0, 3.3, 3.2, 3.1)
)
```

```
data
```

```
##      x1  x2  x3
## 1 2.0 5.1 3.2
## 2 2.1 5.0 3.1
## 3 1.9 5.2 3.0
## 4 2.2 4.9 3.3
## 5 2.0 5.0 3.2
## 6 2.1 5.1 3.1
```

3.1.2 Mean (evaluation point)

We evaluate the model at:

```
mu <- colMeans(data)
mu
```

```
##      x1      x2      x3
## 2.05 5.05 3.15
```

3.1.3 Covariance matrix (estimated from data)

```
Sigma <- cov(data)
Sigma
```

```
##           x1           x2           x3
## x1  0.011 -0.009  0.007
## x2 -0.009  0.011 -0.009
## x3  0.007 -0.009  0.011
```

3.1.4 Model and gradient

Let:

$$I = x_1 x_2 + x_3^2.$$

We compute both the function and its gradient.

```
f <- function(x) {
  list(
    I = x[1] * x[2] + x[3]^2,
    grad = c(
      x[2],      # dI/dx1
      x[1],      # dI/dx2
      2 * x[3]   # dI/dx3
    )
  )
}
```

Then evaluate at the mean point:

```
res <- f(mu)

I_hat <- as.numeric(res$I)
g <- res$grad
```

```
g
```

```
##      x2      x1      x3
## 5.05 2.05 6.30
```

```
I_hat
```

```
## [1] 20.275
```

3.1.5 Variance propagation

$$\text{Var}(I) \approx \nabla f^\top \Sigma \nabla f.$$

```
var_I <- as.numeric(t(g) %*% Sigma %*% g)
var_I
```

```
## [1] 0.78994
```

Standard deviation:

```
sd_I <- sqrt(var_I)
sd_I
```

```
## [1] 0.8887857
```

3.1.6 Final result

```
result <- data.frame(
  quantity = c("mean", "variance", "sd"),
  value = c(round(I_hat, 3), round(var_I, 3), round(sd_I, 3))
)
```

```
result
```

```
##  quantity  value
## 1      mean 20.275
## 2 variance  0.790
## 3        sd  0.889
```

The standard deviation of the output is approximately 0.89, meaning that if the experiment were repeated under the same conditions, the values of I would typically fluctuate by about 0.89 around their mean.

3.1.7 Interpretation

- The **gradient** ∇f describes the *local sensitivity* of the output I to small changes in each input variable. Large values indicate that small changes in that input produce large changes in I .
- The **covariance matrix** Σ describes the *statistical uncertainty structure* of the inputs, including both:
 1. the variability of each input (variances), and
 2. the dependence between inputs (covariances).
- The propagated variance

$$\text{Var}(I) \approx \nabla f^\top \Sigma \nabla f$$

combines these two components by weighting input uncertainties according to the sensitivity of the model. Inputs that are both highly uncertain and highly influential contribute most to the output variance.

4. Idea of bootstrapping + intuition

The previous section approximated the uncertainty of $I = f(x_1, x_2, x_3)$ using a first-order Taylor expansion, leading to the delta method estimator

$$\text{Var}(I) \approx \nabla f^\top \Sigma \nabla f.$$

This approach relies on a local linear approximation of the model and a single evaluation at the mean of the data. While computationally efficient, it may become inaccurate when the model is nonlinear or when the distribution of the inputs is complex.

Instead of deriving uncertainty analytically, an alternative approach is to estimate the distribution of I directly from the data using resampling methods. This leads to the bootstrap approach.

Core idea

The empirical distribution of the sample is used as a proxy for the unknown population distribution.

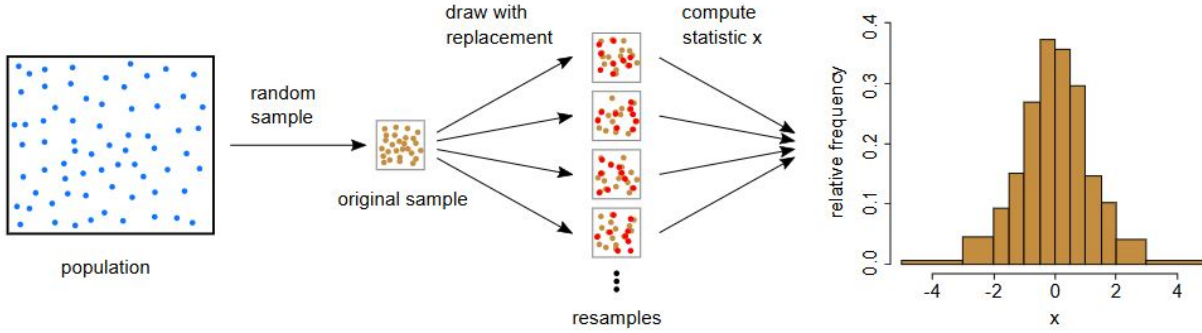


Figure 1: Visual intuition: How bootstrapping works. Source: Wikipedia.

5. Bootstrap algorithm

Given observed data $x = (x_1, \dots, x_n)$:

1. Draw a bootstrap sample x^* of size n **with replacement**
2. Compute the statistic of interest:

$$\hat{\theta}^* = s(x^*)$$

3. Repeat steps 1–2 for B iterations
4. Obtain the bootstrap distribution of $\hat{\theta}$

6. Interpretation of the bootstrap

The bootstrap distribution approximates:

- the sampling distribution of the estimator
- the variability induced by repeated sampling

From this distribution we can estimate:

- standard error
- bias
- confidence intervals

7. Standard error and confidence intervals (Uncertainty measures)

The bootstrap standard error is given by:

$$SE(\hat{\theta}) = SD(\hat{\theta}^*)$$

For asymmetric distributions, percentile-based confidence intervals are used:

$$CI_{95\%} = [q_{0.025}, q_{0.975}]$$

8. Environmental example: uncertainty in a spatial index

We use the Meuse dataset (soil contamination data).

Let $Z_1, \dots, Z_n$ denote observed zinc concentrations and $C_1, \dots, C_n$ observed copper concentrations. We define the environmental index:

$$I = \frac{\bar{Z}}{\bar{C}}, \quad \text{where } \bar{Z} = \frac{1}{n} \sum_{i=1}^n Z_i, \quad \bar{C} = \frac{1}{n} \sum_{i=1}^n C_i$$

Such indices are common in:

- pollution assessment
- ecological risk scoring
- environmental monitoring

8.1 Bootstrap mean (simulated example)

```
set.seed(123)

# -----
# 1. Simulate "population"
# -----
mu_true <- 5
sigma <- 2

# sample (this is what students observe)
x <- rnorm(50, mean = mu_true, sd = sigma)

sample_mean <- mean(x)

# -----
# 2. Bootstrap
# -----
B <- 2000
boot_means <- numeric(B)

for (b in 1:B) {
  x_star <- sample(x, replace = TRUE)
  boot_means[b] <- mean(x_star)
}

boot_mean_est <- mean(boot_means)

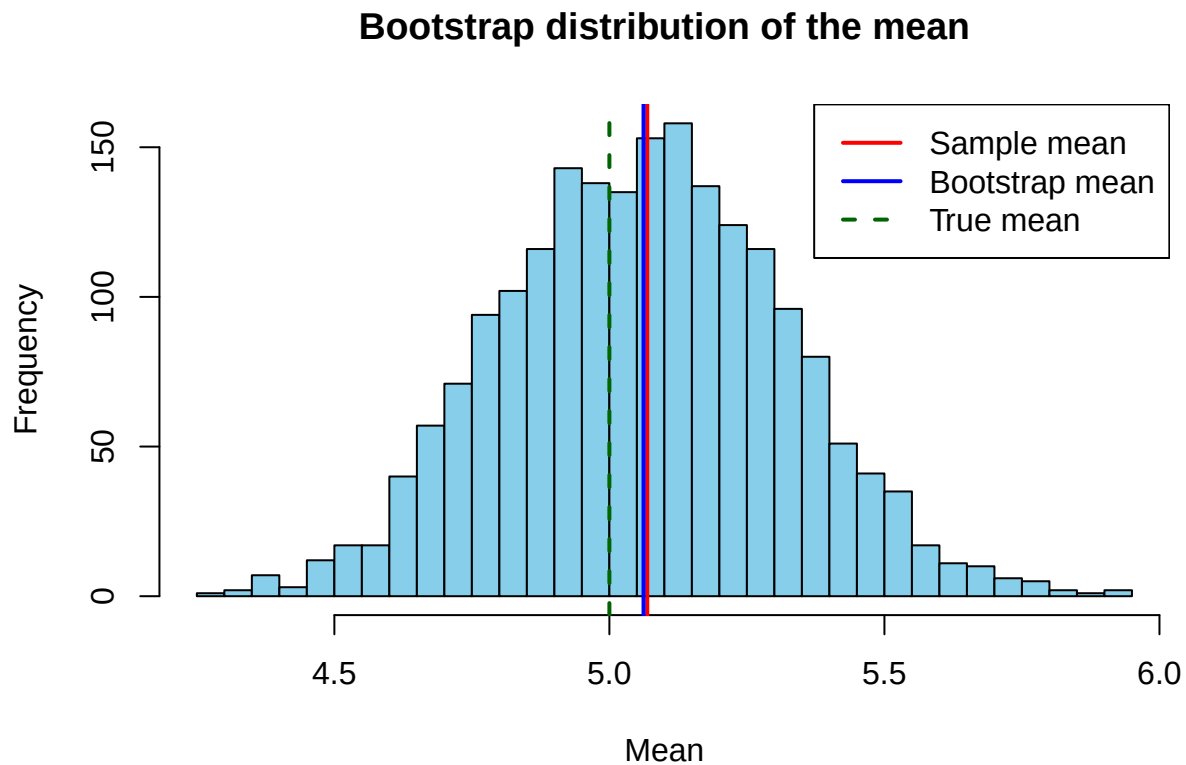
# -----
# 3. Plot distribution
# -----
hist(boot_means,
     breaks = 30,
     col = "skyblue",
     main = "Bootstrap distribution of the mean",
     xlab = "Mean")
```

```

abline(v = sample_mean, col = "red", lwd = 2)
abline(v = boot_mean_est, col = "blue", lwd = 2)
abline(v = mu_true, col = "darkgreen", lwd = 2, lty = 2)

legend("topright",
      legend = c("Sample mean", "Bootstrap mean", "True mean"),
      col = c("red", "blue", "darkgreen"),
      lwd = 2,
      lty = c(1, 1, 2))

```



```
sample_mean
```

```
## [1] 5.068807
```

```
boot_mean_est
```

```
## [1] 5.062082
```

```
sd(boot_means)
```

```
## [1] 0.2560585
```



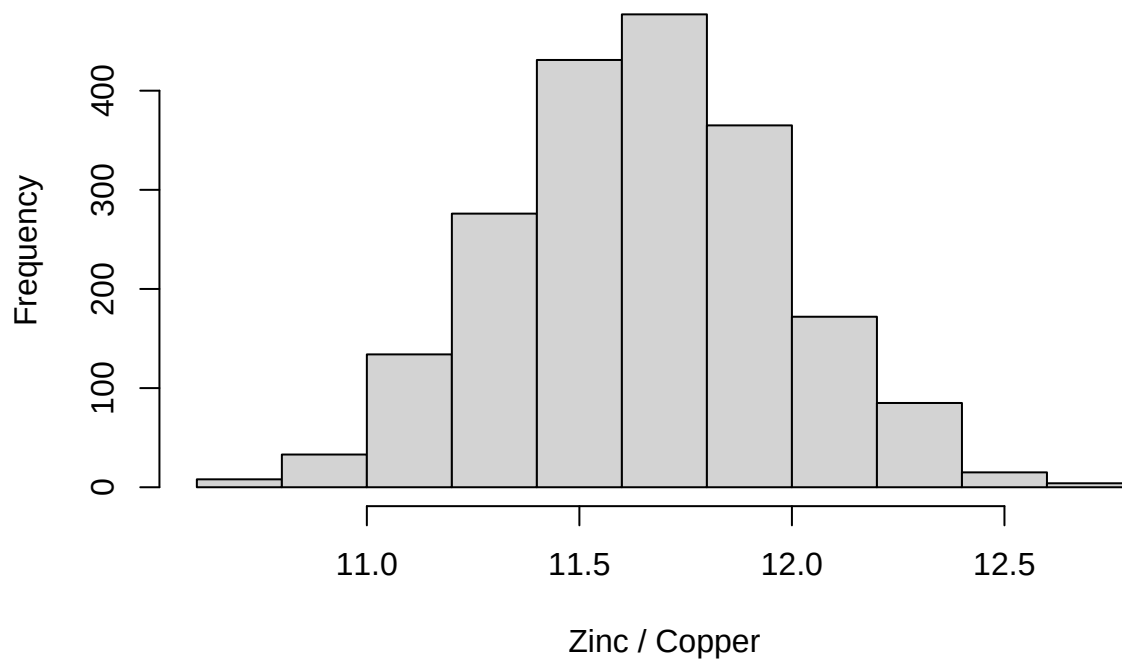
```
quantile(boot_means, c(0.025, 0.975))
```

```
##      2.5%      97.5%  
## 4.576968 5.563031
```

8.2 Bootstrap estimate from meuse data

```
library(sp)  
data(meuse)  
coordinates(meuse) <- ~x+y  
  
zinc <- meuse$zinc  
copper <- meuse$copper  
  
set.seed(123)  
  
B <- 2000  
boot_I <- numeric(B)  
  
for (b in 1:B) {  
  idx <- sample(1:length(zinc), replace = TRUE)  
  boot_I[b] <- mean(zinc[idx]) / mean(copper[idx])  
}  
  
hist(boot_I,  
     main = "Bootstrap distribution of environmental index",  
     xlab = "Zinc / Copper")
```

Bootstrap distribution of environmental index



```
quantile(boot_I, c(0.025, 0.975))
```

```
##      2.5%      97.5%  
## 11.02044 12.29847
```

```
sd(boot_I)
```

```
## [1] 0.3260784
```

8.3 Bootstrap regression model

```
set.seed(123)  
  
df <- data.frame(  
  x = rnorm(50),  
  y = rnorm(50)  
)  
  
B <- 1000  
boot_beta <- numeric(B)  
  
for (b in 1:B) {
```

```

idx <- sample(1:nrow(df), replace = TRUE)
model <- lm(y ~ x, data = df[idx, ])
boot_beta[b] <- coef(model)[2]
}

sd(boot_beta)

```

```
## [1] 0.1124064
```

```
quantile(boot_beta, c(0.025, 0.975))
```

```
##      2.5%      97.5%
## -0.2431516  0.1856379
```

8.4 Bootstrap of a derived index

Consider a nonlinear estimator:

$$I = \frac{\bar{a}}{\bar{b}}$$

```

set.seed(123)

a <- 1:10
b <- runif(10)

B <- 1000
boot_I <- numeric(B)

for (i in 1:B) {
  a_sam <- sample(a, replace = TRUE)
  b_sam <- sample(b, replace = TRUE)
  boot_I[i] <- mean(a_sam) / mean(b_sam)
}

quantile(boot_I, c(0.025, 0.975))

```

```
##      2.5%      97.5%
##  6.021456 15.652390
```

```
sd(boot_I)
```

```
## [1] 2.378926
```

9. When bootstrapping fails

Bootstrapping is powerful, but not universally valid.

It can fail or become unreliable when:

- sample size is very small
- observations are autocorrelated (time series, spatial data)
- distributions are heavy-tailed or unstable

In these cases, standard bootstrap procedures may underestimate uncertainty.

Solution:

To address these issues, several modified resampling methods can be used:

- **Block bootstrap:** used for time series data, where consecutive observations are resampled in blocks to preserve temporal dependence.
- **Spatial bootstrap:** used for spatial data, where resampling respects spatial correlation structures between nearby observations.
- **Stratified bootstrap:** used for heterogeneous populations, where resampling is performed within predefined subgroups to preserve group structure.

9.1 Example: violating independence (time series)

We simulate an autocorrelated process (AR(1)):

```
set.seed(123)

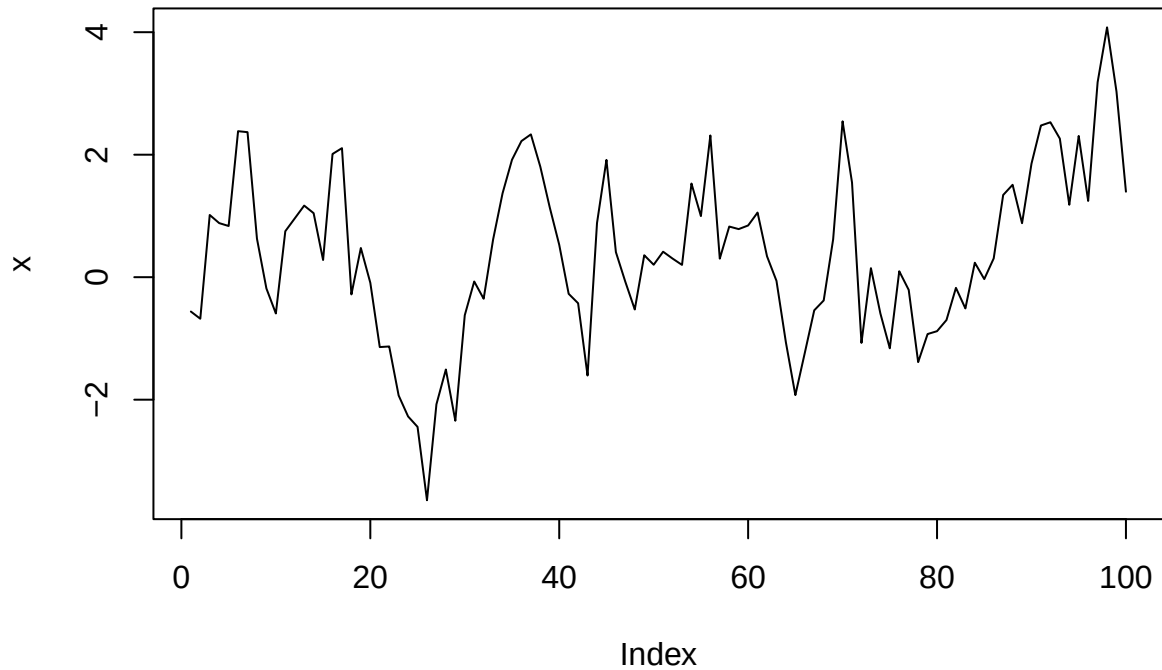
n <- 100

# AR(1) process: strong autocorrelation
x <- numeric(n)
x[1] <- rnorm(1)

for (t in 2:n) {
  x[t] <- 0.8 * x[t - 1] + rnorm(1)
}

plot(x, type = "l", main = "Autocorrelated time series (AR(1))")
```

Autocorrelated time series (AR(1))



9.2 Wrong approach: Naive bootstrap (breaks dependence)

```
set.seed(123)

B <- 1000
boot_means <- numeric(B)

for (b in 1:B) {
  x_star <- sample(x, replace = TRUE)
  boot_means[b] <- mean(x_star)
}

sd_naive <- sd(boot_means)
sd_naive
```

```
## [1] 0.1373441
```

9.3 Correct approach: Block Bootstrap

```
set.seed(123)
```

```

n <- length(x)
B <- 1000
block_size <- 5 # key tuning parameter

boot_means <- numeric(B)

for (b in 1:B) {

  # number of blocks needed
  n_blocks <- ceiling(n / block_size)

  blocks <- vector("list", n_blocks)

  for (i in 1:n_blocks) {

    start <- sample(1:(n - block_size + 1), 1)
    blocks[[i]] <- x[start:(start + block_size - 1)]
  }

  x_star <- unlist(blocks)[1:n]

  boot_means[b] <- mean(x_star)
}

sd_block_boot <- sd(boot_means)
sd_block_boot

```

```
## [1] 0.2469868
```

9.4 Which is better?

We use a Monte Carlo benchmark (simulation truth):

```

# Example

set.seed(123)

M <- 2000
n <- length(x)

means <- numeric(M)

for (m in 1:M) {
  y <- numeric(n)
  y[1] <- rnorm(1)

  for (t in 2:n) {
    y[t] <- 0.8 * y[t - 1] + rnorm(1)
  }

  means[m] <- mean(y)
}

```

```
sd_true <- sd(means)

c(
  naive_bootstrap = sd_naive,
  block_bootstrap = sd_block_boot,
  true_variation = sd_true
)

## naive_bootstrap block_bootstrap true_variation
##      0.1373441      0.2469868      0.4668709
```

This results means that bootstrap underestimates uncertainty when dependence is ignored (naive case).

10. Jackknife

The jackknife is a related resampling technique:

- systematically leave out one observation at a time
- recompute the estimator

For n observations:

$$\hat{\theta}_{(i)} = s(x_{-i})$$

The jackknife can be used to estimate:

- bias
- variance
- standard error (with correction)

```
# Example

set.seed(123)

# Simple dataset
x <- rnorm(30)
y <- 2 * x + rnorm(30)

n <- length(x)

theta_hat <- coef(lm(y ~ x))[2]

jack_theta <- numeric(n)

for (i in 1:n) {
  fit <- lm(y[-i] ~ x[-i])
  jack_theta[i] <- coef(fit)[2]
}
```

```

# Jackknife estimate of SE
theta_bar <- mean(jack_theta)

jack_se <- sqrt((n - 1) / n * sum((jack_theta - theta_bar)^2))

theta_hat

```

```

##           x
## 1.865966

```

```

jack_se

```

```

## [1] 0.1774046

```

11 Comparison of Standard Errors: OLS, Jackknife, and Bootstrap

```

set.seed(123)

# Simulated data (simple but realistic)
n <- 40
x <- rnorm(n)
y <- 2 * x + rnorm(n)

# -----
# 1. Classical OLS SE
# -----
fit <- lm(y ~ x)
ols_se <- summary(fit)$coefficients[2, 2]
ols_beta <- coef(fit)[2]

# -----
# 2. Jackknife SE
# -----
jack_beta <- numeric(n)

for (i in 1:n) {
  fit_i <- lm(y[-i] ~ x[-i])
  jack_beta[i] <- coef(fit_i)[2]
}

jack_mean <- mean(jack_beta)
jack_se <- sqrt((n - 1) / n * sum((jack_beta - jack_mean)^2))

# -----
# 3. Bootstrap SE
# -----
B <- 1000
boot_beta <- numeric(B)

```



```

for (b in 1:B) {
  idx <- sample(1:n, replace = TRUE)
  fit_b <- lm(y[idx] ~ x[idx])
  boot_beta[b] <- coef(fit_b)[2]
}

boot_se <- sd(boot_beta)

# -----
# 4. Comparison
# -----
results <- data.frame(
  Method = c("OLS", "Jackknife", "Bootstrap"),
  SE = c(ols_se, jack_se, boot_se)
)

results

```

```

##      Method      SE
## 1      OLS 0.1731671
## 2 Jackknife 0.1882263
## 3 Bootstrap 0.1767608

```

```

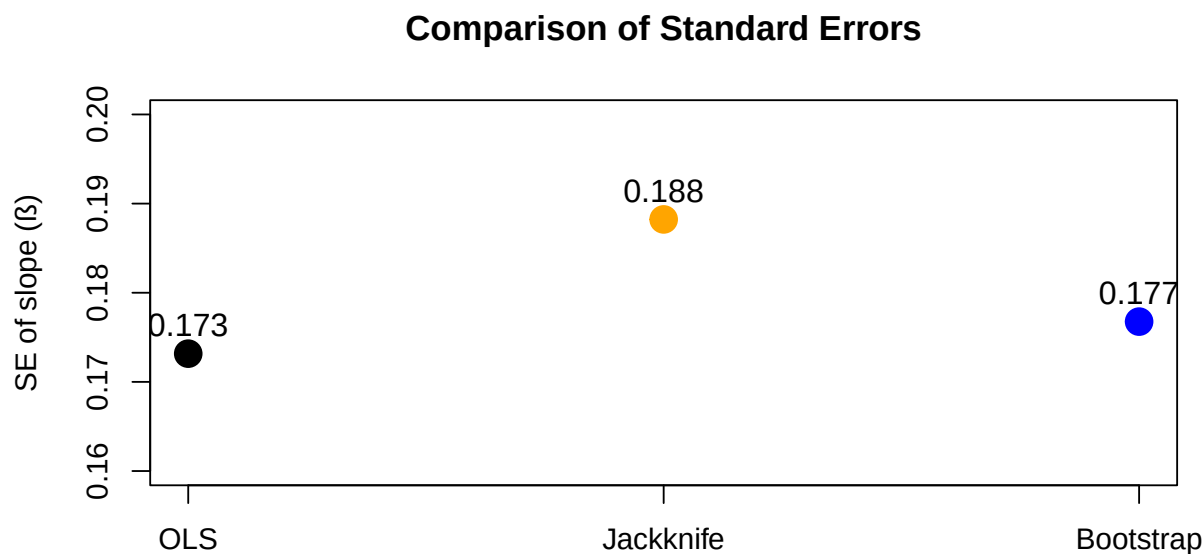
se_values <- c(
  OLS = ols_se,
  Jackknife = jack_se,
  Bootstrap = boot_se
)

plot(
  se_values,
  pch = 19,
  col = c("black", "orange", "blue"),
  cex = 2,
  xaxt = "n", # remove default x-axis
  ylim = c(0.16, 0.20),
  main = "Comparison of Standard Errors",
  ylab = "SE of slope ( )",
  xlab = ""
)

axis(1, at = 1:3, labels = names(se_values))

text(1:3, se_values,
  labels = round(se_values, 3),
  pos = 3,
  cex = 1.1)

```



We compare standard errors obtained from three different approaches: classical OLS theory, the jackknife, and the bootstrap. While OLS relies on model assumptions, both jackknife and bootstrap estimate uncertainty directly from resampling the observed data.

In well-specified linear models, all three methods tend to agree. Larger differences would indicate model misspecification or dependence structures.

12 Summary/Takeaway

Bootstrapping provides a general, simulation-based framework for statistical inference.

Instead of deriving uncertainty analytically, we approximate it by repeatedly resampling the observed data and recomputing the estimator.

It is:

- simple to implement
- flexible for complex statistics
- applicable when analytical solutions fail
- computational rather than assumption-driven

In practice, if you can compute a statistic, you can bootstrap it.